

# 41039 Programming 1

Revision of Week 1: programs, Java, types and errors



Dr. Xuan-Bach Le

`le.xuan.bach@hcmut.edu.vn`

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

## How the next fifteen minutes work

---

**This is not a test.** Nothing here is recorded against you, and nothing you say wrongly costs you anything.

- Every slide **asks** before it **tells**. Commit to an answer first — out loud, or on paper, or just in your head.
- **Answering earns extra credit.** Being wrong out loud earns it too. The only way to miss out is to say nothing.
- If a slide makes no sense to you, that is the single most useful thing you can tell me today. Say so.

**Why bother revising at all?** Everything today — choosing and repeating — is built on top of Week 1. A shaky `int` makes a shaky `for` loop.

**Recalling something is what fixes it. Re-reading it does not.**

# Outline

---

- 1 Java and Python
- 2 Your first Java program
- 3 Variables, types and values
- 4 The same ideas in Python
- 5 When things go wrong
- 6 Live coding
- 7 Bridge to today

## Fill the blanks: Java and Python

- Java is (1) before it runs; Python is (2), line by line.
- Java is (3) typed: types are fixed and checked *before* the program runs. Python is (4) typed: variables have no type — but the data still does.
- Java has four whole-number types and you pick the size. Python has (5), with no size limit.
- Both languages are (6): you never free memory by hand.

### Word bank

- |                      |                    |
|----------------------|--------------------|
| a) interpreted       | g) statically      |
| b) one               | h) procedural      |
| c) compiled          | i) dynamically     |
| d) imperative        | j) four            |
| e) garbage collected | k) object oriented |
| f) two               | l) semicolons      |

**Only the last two rows of the comparison table differ — and they explain nearly everything else you will notice.**

## Fill the blanks: Java and Python

- Java is (1) before it runs; Python is (2), line by line.
- Java is (3) typed: types are fixed and checked *before* the program runs. Python is (4) typed: variables have no type — but the data still does.
- Java has four whole-number types and you pick the size. Python has (5), with no size limit.
- Both languages are (6): you never free memory by hand.

### Answers

- (1) c) **compiled**    (2) a) **interpreted**  
(3) g) **statically**    (4) i) **dynamically**  
(5) b) **one** — Python's `int` has no size limit  
(6) e) **garbage collected**. Procedural, imperative, object oriented: both languages are all three

**Only the last two rows of the comparison table differ — and they explain nearly everything else you will notice.**

## Which kind of error is it?

---

### Compile-time, run-time or logic?

1. You forget a semicolon.
2. `int userName = input.nextInt();`  
— and the user types Bach.
3. Your program averages 74, 75 and 77 and prints 75.
4. You write Bob where you declared bob.
5. In Python, `print(name + age)` where age is an int.

## Which kind of error is it?

### Compile-time, run-time or logic?

1. You forget a semicolon.
2. `int userName = input.nextInt();`  
— and the user types Bach.
3. Your program averages 74, 75 and 77 and prints 75.
4. You write Bob where you declared bob.
5. In Python, `print(name + age)` where age is an int.

### Answers

1. compile-time
2. run-time — `InputMismatchException`. It works fine if your name happens to be a number
3. **logic**
4. compile-time — cannot find symbol
5. run-time — Python's `TypeError`

**Four of the five tell you. Only the logic error is silent — and that one cost Knight Capital \$460 million.**

# Outline

---

- 1 Java and Python
- 2 Your first Java program
- 3 Variables, types and values
- 4 The same ideas in Python
- 5 When things go wrong
- 6 Live coding
- 7 Bridge to today

# Spot the errors

Alone ■ 60 seconds



**X Four things are wrong. The file is Greeting.java.**

```
1 public class greeting {  
2  
3     public static void Main(String[] args) {  
4         System.out.println("Hi there!")  
5         System.out.Println("Bye!");  
6     }  
7 }
```



**X Four things are wrong. The file is Greeting.java.**

```
1 public class greeting {  
2  
3     public static void Main(String[] args) {  
4         System.out.println("Hi there!")  
5         System.out.Println("Bye!");  
6     }  
7 }
```

## ✓ Fixes

- Line 1: the class name must match the file — Greeting, not greeting. Java is **case sensitive**.
- Line 3: the method is main, lowercase. Main compiles, then the program will not start.
- Line 4: every statement ends with a ;
- Line 5: println, not Println. Case again.

**bob != Bob != BOB. This will cause most of your errors for another fortnight.**

## Fill the blanks: name the parts

```
public (1) Hello {  
  
    public static (2) main(  
        (3) [] args) {  
  
        System.out. (4) (  
            "Hello 41039!") (5)  
    }  
}
```

And: this file must be called (6)

### Word bank

- |           |               |
|-----------|---------------|
| a) void   | g) Hello.java |
| b) print  | h) class      |
| c) ;      | i) int        |
| d) String | j) hello.java |
| e) method | k) println    |
| f) :      | l) Main.java  |

# Fill the blanks: name the parts

```
public (1) Hello {

    public static (2) main(
        (3) [] args) {

        System.out. (4) (
            "Hello 41039!") (5)
    }
}
```

And: this file must be called (6)

## Answers

(1) h) class — everything from { to the matching } belongs to it

(2) a) void — main is a **method**:  
<return type>  
<name>(<parameters>); void means it returns nothing

(3) d) String — the type of the parameter

(4) k) println — print would leave the cursor on the line

(5) c) ; — every statement ends with one

(6) g) Hello.java — the class name, case included

# Outline

---

- 1 Java and Python
- 2 Your first Java program
- 3 Variables, types and values**
- 4 The same ideas in Python
- 5 When things go wrong
- 6 Live coding
- 7 Bridge to today

# Types and names: quick quiz

---

## A. Which type would you declare?

```
---- studentCount = 341;  
---- average      = 75.33;  
---- grade        = 'D';  
---- passed       = true;
```

## B. Which break Java's naming conventions?

```
class Student    class student  
int MyTotal      int myTotal    Math.PI
```

## C. What is the biggest number an int holds?

# Types and names: quick quiz

## A. Which type would you declare?

```
---- studentCount = 341;  
---- average      = 75.33;  
---- grade        = 'D';  
---- passed       = true;
```

## B. Which break Java's naming conventions?

```
class Student    class student  
int MyTotal      int myTotal    Math.PI
```

## C. What is the biggest number an int holds?

### Answers

**A.** int, double, char, boolean. Note 'D' uses single quotes — "D" is a String.

**B. ~~X~~ Wrong:** class student — class names start **uppercase** — and int MyTotal — variables start **lowercase**. Math.PI is a constant, so all caps is right. Conventions, not rules: the compiler accepts every one of them.

**C.** 2,147,483,647 — that is  $2^{31} - 1$ , and it is why an int broke YouTube's view counter.

**In practice: int, double, char, boolean.**

# Predict the output

Alone ■ 60 seconds



```
int a = 7;
int b = 2;

System.out.println(a / b);      // (a)
System.out.println(7.0 / b);   // (b)
System.out.println(a % b);     // (c)
System.out.println("a = " + a); // (d)
System.out.println(a + b);     // (e)

a = a + 1;
System.out.println(a);         // (f)
```

# Predict the output

Alone ■ 60 seconds

```
int a = 7;
int b = 2;

System.out.println(a / b);      // (a)
System.out.println(7.0 / b);    // (b)
System.out.println(a % b);      // (c)
System.out.println("a = " + a); // (d)
System.out.println(a + b);      // (e)

a = a + 1;
System.out.println(a);          // (f)
```

## Answers

- (a) 3 — int divided by int gives an **int**. The .5 is not rounded, it is *discarded*
- (b) 3.5 — one double operand is enough
- (c) 1 — % is the remainder
- (d) a = 7 — here + **joins Strings**
- (e) 9 — the same +, now **adding**. What it does depends on the types beside it
- (f) 8 — read = right to left: *take a+1, put it in a*

**= is not equality. It is "take the right, store it in the left".**

# The bug that compiles, runs, and lies

---

```
int totalMarks = 226;  
int studentCount = 3;  
  
int classAverage  
    = totalMarks / studentCount;  
  
System.out.println("Average: "  
    + classAverage);
```

## Three questions

1. What does it print?
2. What *should* it print?
3. Which of the three kinds of error is this?

# The bug that compiles, runs, and lies

```
int totalMarks = 226;
int studentCount = 3;

int classAverage
    = totalMarks / studentCount;

System.out.println("Average: "
    + classAverage);
```

## Three questions

1. What does it print?
2. What *should* it print?
3. Which of the three kinds of error is this?

## Answers

1. Average: 75
2. 75.33
3. A **logic error**. It compiles, it runs, and it prints a plausible number every single time.

✓ **Fix** (double) totalMarks / studentCount — and make classAverage a double too. Changing only the variable is not enough: the division has already happened.

**One wrong type. No error message. This is the class of bug that makes the news.**

# Outline

---

- 1 Java and Python
- 2 Your first Java program
- 3 Variables, types and values
- 4 The same ideas in Python**
- 5 When things go wrong
- 6 Live coding
- 7 Bridge to today

# Same program, two languages: name five differences

---

```
public class Greet {  
    public static void main(String[] a) {  
        String name = "Bach";  
        int age = 20;  
        boolean ok = true;  
        System.out.println(name + age);  
    }  
}
```

```
name = "Bach"  
age = 20  
ok = True  
print(name + str(age))
```

Why does the Python version need `str(age)` when the Java one does not?

## Same program, two languages: name five differences

```
public class Greet {  
    public static void main(String[] a) {  
        String name = "Bach";  
        int age = 20;  
        boolean ok = true;  
        System.out.println(name + age);  
    }  
}
```

```
name = "Bach"  
age = 20  
ok = True  
print(name + str(age))
```

Why does the Python version need `str(age)` when the Java one does not?

### Answers

No class or main; no type on the left of `=`; no semicolons; no braces; True is capitalised where Java writes true. And the `str()`: Java's `+` silently converts the `int`; Python's *refuses* to mix `str` and `int`, and raises a `TypeError`.

**In Python variables have no type. The data still does. That is “dynamically typed”.**

# One space changes the answer

**A**

```
a = 20
if a < 10:
    print("small")
    print("done")
```

**B**

```
a = 20
if a < 10:
    print("small")
print("done")
```

**With `a = 20`, what does each print?**

Only the indentation of the last line differs.

# One space changes the answer

**A**

```
a = 20
if a < 10:
    print("small")
    print("done")
```

**B**

```
a = 20
if a < 10:
    print("small")
print("done")
```

**With `a = 20`, what does each print?**

Only the indentation of the last line differs.

## Answers

**A** prints **nothing** — both prints are inside the `if`.

**B** prints `done` — that line is outside it, so it always runs.

Four spaces, never mixed with tabs.

**In Java bad indentation is ugly. In Python it is wrong.**

# Outline

---

- 1 Java and Python
- 2 Your first Java program
- 3 Variables, types and values
- 4 The same ideas in Python
- 5 When things go wrong**
- 6 Live coding
- 7 Bridge to today

# Read the compiler, do not argue with it

---

```
Greeting.java:4: error: cannot find symbol
    System.out.println("Hi " + userName);
                        ^
    symbol:   variable userName
    location: class Greeting
1 error
```

## Four questions

1. Which file, and which line?
2. Which token?
3. What does cannot find symbol actually mean?
4. You get 14 errors. Which do you fix?

# Read the compiler, do not argue with it

```
Greeting.java:4: error: cannot find symbol
    System.out.println("Hi " + userName);
                        ^
symbol:   variable userName
location: class Greeting
1 error
```

## Four questions

1. Which file, and which line?
2. Which token?
3. What does cannot find symbol actually mean?
4. You get 14 errors. Which do you fix?

## Answers

1. Greeting.java, line 4
  2. userName — follow the ^
  3. *You used a name I have never seen* — a typo, a missing declaration, or a capitalisation slip
  4. The **first** one, then recompile. Errors 2–14 are usually downstream of error 1
- And turn on line numbers in Ed — without them, Greeting.java:4 tells you nothing.

**Which file, which line, what kind, which token. Every time, in that order.**

## In pairs: make it print Bach20

Pairs ■ 2 minutes

**X This crashes. Why, and what is the smallest fix?**

```
name = "Bach"  
age = 20  
print(name + age)
```

### Then answer this

The equivalent Java line, `System.out.println(name + age)`, does *not* crash. Is Java being helpful, or is it being dangerous?

## In pairs: make it print Bach20

Pairs ■ 2 minutes

**X This crashes. Why, and what is the smallest fix?**

```
name = "Bach"  
age = 20  
print(name + age)
```

### Then answer this

The equivalent Java line, `System.out.println(name + age)`, does *not* crash. Is Java being helpful, or is it being dangerous?

✓ **Fix**

```
print(name + str(age))
```

`TypeError: can only concatenate str (not "int") to str`. Python will not guess what you meant.

### Both, honestly

Java's convenience is real, and so is the risk: the day you *meant* to add two numbers and one of them was a `String`, Java will happily join them and say nothing. Python's refusal is noisy, early and cheap.

# Outline

---

- 1 Java and Python
- 2 Your first Java program
- 3 Variables, types and values
- 4 The same ideas in Python
- 5 When things go wrong
- 6 Live coding**
- 7 Bridge to today

```
import java.util.Scanner;

public class Average {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("First number: ");
        int a = input.nextInt();
        System.out.print("Second number: ");
        // TODO 1: read the second number into an
        int b

        // TODO 2: an int sum, holding a + b

        // TODO 3: a double average. 7 and 2 give
        4.5

        System.out.println("Sum: " + sum);
        System.out.println("Average: " + average);
    }
}
```

### Expected run

```
First number: 7
Second number: 2
Sum: 9
Average: 4.5
```

One volunteer types, on the projector.  
Everyone else: shout the moment you  
see a mistake — that earns credit  
too. Run it before anyone says it is  
finished.

```
import java.util.Scanner;

public class Average {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("First number: ");
        int a = input.nextInt();
        System.out.print("Second number: ");
        // TODO 1: read the second number into an
        int b

        // TODO 2: an int sum, holding a + b

        // TODO 3: a double average. 7 and 2 give
        4.5

        System.out.println("Sum: " + sum);
        System.out.println("Average: " + average);
    }
}
```

### Expected run

```
First number: 7
Second number: 2
Sum: 9
Average: 4.5
```

One volunteer types, on the projector.  
Everyone else: shout the moment you  
see a mistake — that earns credit  
too. Run it before anyone says it is  
finished.

### Watch for

sum / 2 compiles, runs, and

# One answer — and then break it

## ✓ The three missing lines

```
int b = input.nextInt();  
  
int sum = a + b;  
  
double average = sum / 2.0;  
// or: (double) sum / 2
```

## ✗ The one that lies

```
double average = sum / 2; // 4, not 4.5
```

`int / int` is an `int`. Storing it in a `double` afterwards does not bring the `.5` back.

# One answer — and then break it

## ✓ The three missing lines

```
int b = input.nextInt();  
  
int sum = a + b;  
  
double average = sum / 2.0;  
// or: (double) sum / 2
```

## ✗ The one that lies

```
double average = sum / 2; // 4, not 4.5
```

`int / int` is an `int`. Storing it in a `double` afterwards does not bring the `.5` back.

### Now break it

Run it again and type seven for the first number.

- What kind of error is this — compile-time, run-time or logic?
- Five lines of stack trace. Which one is *yours*?

# One answer — and then break it

## ✓ The three missing lines

```
int b = input.nextInt();  
  
int sum = a + b;  
  
double average = sum / 2.0;  
// or: (double) sum / 2
```

## ✗ The one that lies

```
double average = sum / 2; // 4, not 4.5
```

`int / int` is an `int`. Storing it in a `double` afterwards does not bring the `.5` back.

## Now break it

Run it again and type seven for the first number.

- What kind of error is this — compile-time, run-time or logic?
- Five lines of stack trace. Which one is *yours*?

## Answers

Run-time: `InputMismatchException`, the same one as Lecture 1. Four lines are inside `Scanner`; the one that says `Average.main` is yours. Start there.

## Live coding: the same, in Python A different volunteer ■ 4 min

```
a = input("First number: ")
b = input("Second number: ")

# TODO 1: a and b are text right now.
#         Turn both into whole numbers.

# TODO 2: total = a + b, and average = total / 2

# TODO 3: print two lines, exactly:
#         Sum: 9    and    Average: 4.5
#         careful: you cannot + a str and an int
```

### Expected run

```
First number: 7
Second number: 2
Sum: 9
Average: 4.5
```

Same rules. Same program as the Java one — so before typing anything: what do you expect to be *different*?

## Live coding: the same, in Python A different volunteer ■ 4 min

```
a = input("First number: ")
b = input("Second number: ")

# TODO 1: a and b are text right now.
#         Turn both into whole numbers.

# TODO 2: total = a + b, and average = total / 2

# TODO 3: print two lines, exactly:
#         Sum: 9    and    Average: 4.5
#         careful: you cannot + a str and an int
```

### Expected run

```
First number: 7
Second number: 2
Sum: 9
Average: 4.5
```

Same rules. Same program as the Java one — so before typing anything: what do you expect to be *different*?

### Watch for

Skip TODO 1 and `a + b` is "72". No error. `input()` always

# One answer — and what was different

---

## ✓ One way

```
a = int(a)
b = int(b)
total = a + b
average = total / 2
print("Sum: " + str(total))
print("Average: " + str(average))
```

## ✗ Skip the int() and nothing complains

```
total = a + b      # "72" -- a logic bug
```

# One answer — and what was different

## ✓ One way

```
a = int(a)
b = int(b)
total = a + b
average = total / 2
print("Sum: " + str(total))
print("Average: " + str(average))
```

## ✗ Skip the int() and nothing complains

```
total = a + b      # "72" -- a logic bug
```

## Three differences, all from Lecture 1

**Input arrives as text.** Java's `nextInt()` converted for you; Python's `input()` does not, so you call `int()` — and typing seven now fails *there*, with a `ValueError`.

**/ gives 4.5 by itself.** Java's `9 / 2` gave 4. Same symbol, different bet.

**+ will not mix.** `"Sum: " + total` is the Bach20 crash again, so `str()` goes back in.

# Outline

---

- 1 Java and Python
- 2 Your first Java program
- 3 Variables, types and values
- 4 The same ideas in Python
- 5 When things go wrong
- 6 Live coding
- 7 Bridge to today**

# Where Week 1 stops

---

## Everything you can write so far

```
int mark = 74;  
  
System.out.println("Mark: " + mark);  
System.out.println("Result: PASS");
```

Top to bottom. It prints PASS for a mark of 74 — and PASS for a mark of 4.

### Two questions for today

1. How would you print FAIL below 50?
2. How would you do it for *all* 341 students?

# Where Week 1 stops

## Everything you can write so far

```
int mark = 74;

System.out.println("Mark: " + mark);
System.out.println("Result: PASS");
```

Top to bottom. It prints PASS for a mark of 74 — and PASS for a mark of 4.

### Two questions for today

1. How would you print FAIL below 50?
2. How would you do it for *all* 341 students?

## ✓ After today

```
for (int mark : marks) {
    if (mark >= 50) {
        System.out.println("PASS");
    } else {
        System.out.println("FAIL");
    }
}
```

1. if-else — branching.
  2. A loop — repetition.
- Both arrive today.

**State was one third of the toolkit. Today you get the other two thirds.**

# You should now be able to...

## Week 1

- ✓ name the **three tools** — state, branching, repetition — and say which one you have
- ✓ write the skeleton of a Java program: class, main, braces, semicolons, matching **file name**
- ✓ declare int, double, char, boolean with a legal name
- ✓ predict **integer division**, %, and + on Strings
- ✓ read = right to left
- ✓ write the same program in **Python**, and say why the indentation is not decoration
- ✓ tell **compile-time**, **run-time** and **logic** errors apart, and say which one is silent
- ✓ read a compiler message: file, line, kind, token — and fix the **first** one

### If any box is unticked

A 20-question quiz with an explanation for every option:

[lexuanbach.github.io/41039](https://lexuanbach.github.io/41039)